Git, GitHub, and VS Code

Agentic AI for Project Management and Research Productivity

Tomas Larroucau

Arizona State University

Monday, October 26, 2026

Workshop Overview

1. Introduction
2. VS Code and Agentic AI
3. Git: Version Control Fundamentals
4. GitHub: Collaboration Platform
5. Advanced Topics and Integrations
6. Next Steps and Questions

Why This Workshop?

The Problem:

- Research projects grow messy fast
- Co-authoring via email and Dropbox can be chaotic
- AI tools exist but live *outside* the workflow

What If We Could:

- Work in one environment: code, writing, collaboration
- Track changes and recover earlier versions
- Let AI operate *inside* the project with guardrails

Goal: Set up the infrastructure for truly integrated, AI-assisted research.

"We'll let the AI roam free... but set up proper guardrails."

The "Jur-AI-ssic Park" Approach to Research

The Park

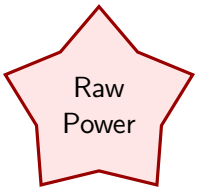
(VS Code)



The island where everything lives

The Dinosaurs

(AI Agents)



Powerful, fast, but chaotic

The Guardrails

(Git & GitHub)



Keeps the chaos contained

"Your scientists were so preoccupied with whether or not they could, they didn't stop to think if they should... **Git lets you undo if they shouldn't.***"*

The Symbiosis: Agents as Builders



From Safety to Superpowers:

- **VS Code → Capability**
  Agents gain *hands*. They can execute tests, debug, and browse the web.
- **Git/GitHub → Memory**
  Issues & PRs provide persistent context and documentation.
- **Agents → Mastery**
  They lower the cost of learning the tools.
  "How do I cherry-pick this commit?"

Reproducible Research Template

Repository Structure:

- `data/`: Sample data for demonstration
- `scripts/`: Analysis, plotting, and table generation
- `output/`: Generated figures and tables
- `tex/`: LaTeX templates (paper, slides)
- `Makefile`: Automation workflow
- `README.md`: Project documentation and agent instructions
- `.gitignore`: Excluded files

Key Principle: Everything generated from scripts, nothing manual!

`https://github.com/tlarroucau/AI_workshop`

See Makefile slide (Appendix)

Workshop Prerequisites

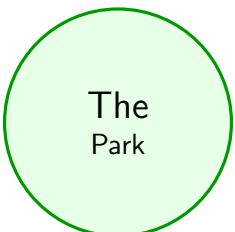
1. **Accounts**
   - **GitHub**: Create an account at `github.com`
   - **Education Benefits**: Check eligibility and activate your benefit at `education.github.com`
2. **Software**
   - **VS Code**: `code.visualstudio.com`
   - **Git**: `git-scm.com`
   - *To run the template*: Python, LaTeX, and Make
3. **Extensions & Setup**
   - Sign in to GitHub within VS Code
   - Set up one agent: Copilot, Codex, or Claude Code
   - Sign in and test access before the workshop
   - *Optional*: GitHub MCP integration

Note on Access

Copilot, provider subscriptions, and API billing have different access and usage limits. Education verification and activation can take time.

The Environment

Visual Studio Code



What is VS Code?

Visual Studio Code

- Free, open-source editor by Microsoft
- Cross-platform (Windows, Mac, Linux)
- Extensible via marketplace
- Integrated terminal
- Built-in Git support
- AI assistant integration

Why VS Code for Research?

- Read, write, and edit code and papers in one place
- Manage entire project lifecycle
- Collaborate seamlessly
- Automate repetitive tasks
- Leverage AI for productivity

Similar AI Editors: Cursor, Windsurf, Antigravity

Useful Extensions for Research & Productivity

Extension	Purpose
AI & Productivity	
GitHub Copilot	AI code assistant
Claude Code	AI coding agent
Codex	OpenAI assistant
Web Search	Copilot web search
VS Code Speech	Speech-to-text input
Data Science	
Python	Language support
Pylance	IntelliSense, type check
Jupyter	Notebook support
R	R language support
Stata Enhanced	Stata syntax + run
Documentation	
LaTeX Workshop	Compile LaTeX
LaTeX Language	Syntax & snippets
LTeX	Grammar/spell check
Markdown All in One	Markdown preview

Extension	Purpose
Version Control & GitHub	
GitHub PR & Issues	Manage PRs/issues
Git History	View git log
GitHub Actions	CI/CD workflows
Data & Utilities	
Rainbow CSV	CSV colorization
Excel Viewer	View Excel files
Code Snap	Code screenshots
Auto Align	Align code formatting
Development Tools	
Remote SSH	Remote development
Live Server	Local web server
Makefile Tools	Makefile support

Essential VS Code Features

Navigation & Interface:

- **Ctrl/Cmd+Shift+P**: Command Palette
- **Ctrl/Cmd+P**: Quick file open
- **Ctrl+‘**: Toggle terminal

Multi-Cursor Magic:

- **Ctrl/Cmd+D**: Select next occurrence
- **Alt+Click**: Add cursor anywhere
- **Ctrl/Cmd+Shift+L**: Select all occurrences

Code Editing:

- **Alt+↑/↓**: Move line up/down
- **Ctrl/Cmd+/**: Toggle comment
- **Ctrl/Cmd+Shift+K**: Delete line
- **Ctrl/Cmd+Shift+D**: Duplicate line

Code Navigation:

- **Ctrl/Cmd+Click**: Go to definition
- **Alt+←/→**: Navigate back/forward

VS Code Workspace Setup

Recommended Workspace Structure:

- Open entire project folder as workspace
- Configure settings per workspace (Python path, linters, etc.)
- Save workspace file (.code-workspace) for team sharing

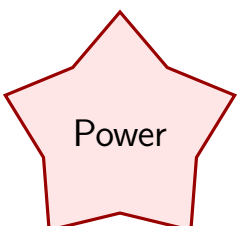
Settings Sync:

- Sync extensions and settings across machines
- Use GitHub or Microsoft account
- Maintain consistency across your own environments

- **Agent panel**: Supported local Claude Code and Codex sessions can appear in the sessions list; enable the *External* filter when available.

Raw Power

AI Agents in Action



What is Agentic AI?

Traditional AI Assistants

- Respond to queries
- Generate code snippets
- Provide suggestions

Agentic AI

- Autonomous task completion
- Multi-step reasoning
- Context-aware assistance

Where Do You Place Yourself?

Research question $\rightarrow$ Project design $\rightarrow$ Methods and tasks $\rightarrow$ Exact code

- Choose how closely to guide each task
- You own the question, assumptions, and interpretation

Codex and Claude Code: Harness Workflows

Model: reasoning. **Harness**: context, tools, permissions. **Interface**: where you work.

Both: project instructions, reusable skills, MCP/tools, hooks, permission controls, and output guidance.

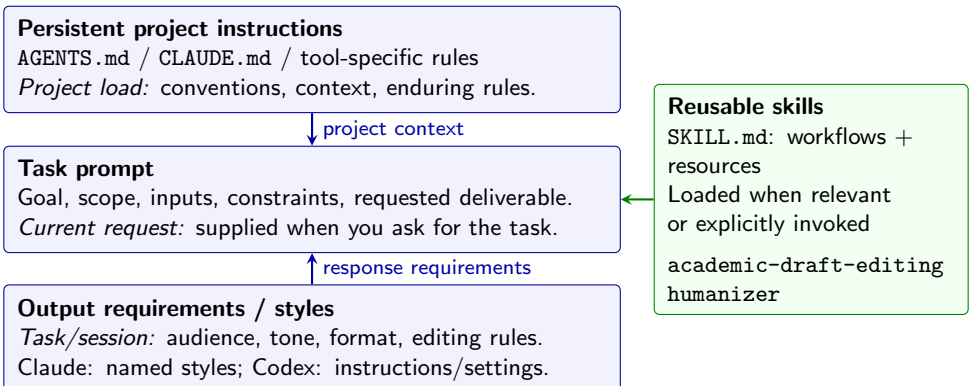
	Claude Code	**OpenAI Codex**
Shape behavior	Named output styles; JS/TS mods in plugins customize session behavior/UI.	Instructions and supported personality settings; skills/hooks for repeated workflows.
Control actions	Tool/path rules and permission modes.	Approval policy and sandbox scope configured separately.
Continue remotely	`claude remote-control`: continue from browser/mobile; keep the host online.	Desktop pairing (Mac/Windows); experimental `codex remote-control` also exists.

Custom mods add code to maintain; support varies by version, plan and platform.

- **GitHub Copilot**: integrated editor/GitHub workflow, with model choice.

Custom Context & Skills

Conceptual organization; loading and precedence depend on the tool.



Agents can help install/use skills; check each tool's supported locations and discovery rules.

Agent Interfaces: CLI, Editor, and Desktop

Terminal / CLI

- Work beside shell commands and scripts
- Useful on a remote host through SSH
- Navigate files and changes through terminal tools

IDE Extension

- Read files and inspect changes beside the conversation
- Useful for close editing and review
- Features depend on the extension and integration

Desktop

- Manage sessions across tasks and projects
- Useful for delegation and oversight
- Remote control requires a running host and supported access

Data Privacy & Security Considerations

Follow Your Institution's Guidelines:

- Always comply with ASU (or your institution's) data security policies
- Review AI tool terms of service for data retention policies!
- Be cautious with sensitive, proprietary, or confidential data

When Cloud AI is Not an Option:

- If data privacy or code confidentiality is a major concern
- If institutional policies prohibit cloud-based AI tools
- If working with regulated data (HIPAA, FERPA, etc.)

Fully Local AI Solutions:

- **Continue** + **Ollama**: Run models on your machine
- **Cline** + local models: Configure an agent with a local model
- Check model hosting, network tools, extensions, and telemetry
- Performance depends on the model, task, and available hardware

The Guardrails (Local)

Git Version Control



What is Git?

- **Distributed version control system**
- Tracks changes to files over time
- Helps coordinate changes with collaborators
- Essential for reproducible research

Why Git for Research?

- History of your committed work
- Experiment safely with branches
- Collaborate with co-authors
- Publish code alongside papers
- Recover from mistakes

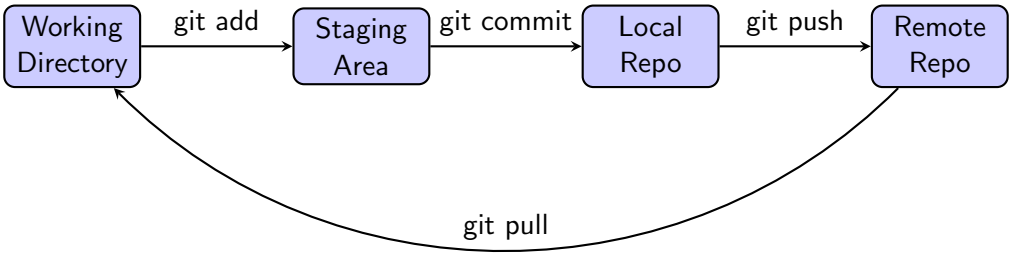
Git Core Concepts

1. **Repository (repo)**: Project folder tracked by Git
2. **Commit**: Snapshot of your project at a point in time
3. **Branch**: Parallel version of your repository
4. **Merge**: Combine changes from different branches
5. **Remote**: Repository hosted online (e.g., GitHub)

Basic Workflow

```
git add file.py          # Stage changes
git commit -m "message"  # Save snapshot
git push                 # Upload to remote
```

The Git Workflow



- Edit files in working directory
- Stage changes you want to commit
- Commit creates permanent snapshot
- Push shares with collaborators

Essential Git Commands

Repository Setup

```
git init                    # Initialize new repo
git clone <url>             # Copy remote repo
```

Daily Workflow

```
git status                  # Check current state
git add <file>              # Stage specific file
git add .                   # Stage all changes
git commit -m "msg"         # Commit with message
git fetch                   # Download remote changes
git pull                    # Fetch + integrate changes
git push origin main        # Push to remote
git log                     # View commit history
git diff                    # See unstaged changes
```

The .gitignore File

What is .gitignore?

- Tells Git which untracked files to ignore
- Helps exclude generated files, credentials, or large binaries
- Usually at the root; subdirectories can have their own

Example .gitignore for Research Projects:

```
# Python
__pycache__/
*.pyc
.ipynb_checkpoints/
*.egg-info/

# R
.Rhistory
.RData
.Rproj.user/
```

```
# LaTeX
*.aux
*.bbl
*.blg
*.log
*.out
*.fls
*.synctex.gz

# Generated outputs
output/
*.pdf
*.png
```

Git Best Practices

Commit Messages

- Be descriptive: "Add regression analysis for Model 2"
- Use imperative mood: "Fix data loading bug"
- Reference issues: "Closes #42"

What to Commit

- DO: Source code, scripts, documentation
- DO: Raw sample data (if reasonable size)
- DON'T: Unneeded generated outputs (rebuild from scripts)
- DON'T: Large binary files (use Git LFS if needed)
- DON'T: Passwords or API keys

Branching Strategy

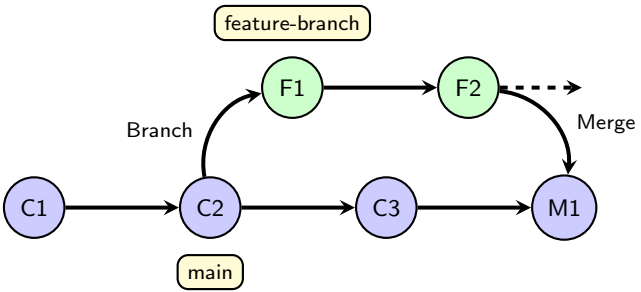
Why branches?

- Isolate experimental work
- Develop features independently
- Keep main branch stable

Common workflow:

1. Create branch for new feature: `git checkout -b feature-name`
2. Make changes and commit
3. Push branch: `git push -u origin feature-name`
4. Create pull request on GitHub
5. Review and merge
6. Delete branch after merge

Visualizing Branches



- **Branch**: Create a parallel timeline (`git checkout -b`)
- **Commit**: Work on the feature branch (F1, F2)
- **Merge**: Combine feature history back into main (`git merge`)
- *Note: The branch remains open after merging unless deleted*

Git Worktrees: Separate Folders for Parallel Work

What is a Worktree?

- Another working folder attached to the same repository
- Check out a different branch in each folder
- Shared Git history; separate files and staging areas

Working with AI Agents:

- One named session and worktree per task
- Parallel agents edit separate working copies
- Review and merge their branches when ready

Basic Commands:

Create a Branch and Working Folder

```
git worktree add -b task-a ../task-a
```

List Working Folders

```
git worktree list
```

Remove a Clean Working Folder

```
git worktree remove ../task-a
```

Common Challenges & Solutions

Understanding Merge Conflicts:

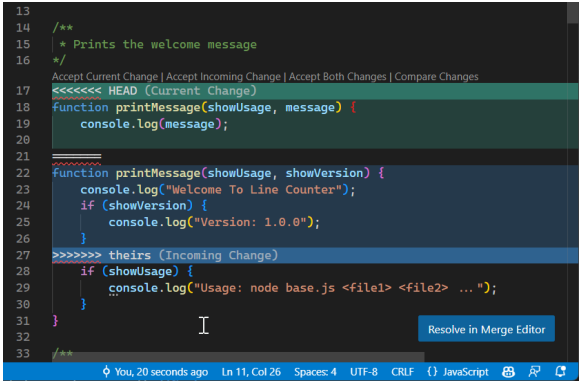
- Occur when Git cannot automatically determine which version to keep
- Happen when the same lines of code/text are edited in different branches
- VS Code highlights conflicts and lets you choose which version to keep
- AI agents can suggest the best resolution strategy!

Backing Up Your Work:

- You can keep your local repo in Dropbox for automatic backup
- Avoid sharing that Dropbox folder with co-authors unless you can really coordinate well

Challenge	Solution
Merge conflicts	Use VS Code conflict resolver, ask AI agent for help
Large files	Use .gitignore, Git LFS if needed
Slow Git operations	Use .gitignore for generated files
Lost work	Commit often, use branches
Unclear AI output	Refine prompts, add context
Reproducibility	Use Makefile, document dependencies

Resolving Merge Conflicts in VS Code



- VS Code shows **conflict markers**: `<<<<<<< HEAD` (yours) vs `>>>>>>> theirs`
- One-click options: *Accept Current*, *Accept Incoming*, *Accept Both*, or *Compare*
- Use **Resolve in Merge Editor** for a side-by-side 3-way diff

The Guardrails (Cloud)

GitHub Collaboration



What is GitHub?

GitHub is...

- Web-based Git hosting
- Social coding platform
- Project management tools
- Collaboration infrastructure
- Portfolio for researchers

Key Features:

- Remote repository hosting
- Pull requests
- Issues and project boards
- Hosts your personal Web site!
- Pages (documentation)
- Copilot (AI Agents)

Education benefits for eligible students and teachers (github.com/education)

Demo: Showing Github and Creating Personal Website

GitHub Workflow: Issues

What are Issues?

- Track tasks, bugs, feature requests
- Organize work with labels and milestones
- Assign to team members
- Reference in commits and PRs

Issue-Driven Development:

1. Create issue: "Add robustness checks"
2. Create branch: `git checkout -b issue-42-robustness`
3. Work on feature, commit with "Addresses #42"
4. Create pull request
5. Merge and close: "Closes #42"

GitHub Project Boards

Organize Research Projects:

- Kanban-style boards
- Columns: To Do, In Progress, Done
- Link to issues and PRs
- Automate card movement

Example Workflow:

1. Create project board for paper
2. Add columns for analysis, writing, revisions
3. Create issues for each task
4. Move cards as work progresses
5. Track progress visually

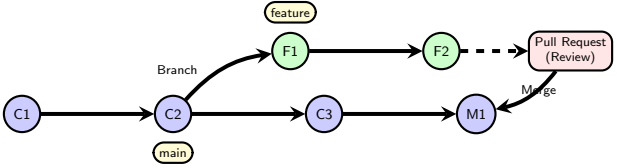
Demo: Showing a project board.

GitHub Workflow: Pull Requests

What are Pull Requests (PRs)?

- Propose changes to repository
- Enable code review before merging
- Discuss implementation details

PR Workflow:



1. Create and push feature branch
2. Open Pull Request on GitHub
3. Review, discuss, and fix
4. Merge when approved

Integrations

Putting It All Together



MCP: Model Context Protocol

What is MCP?

- Protocol that gives AI agents access to external tools and services
- Enables *direct interaction* with APIs

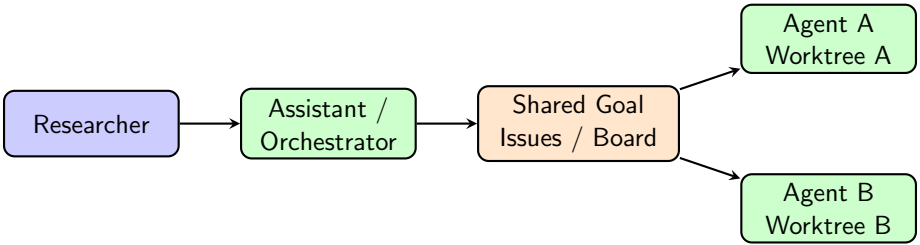
Key MCP Servers:

- **GitHub**: Create issues, PRs, manage boards
- **Playwright**: Browser automation and interaction
- **Context7**: Up-to-date library documentation
- **Firecrawl**: Web scraping and data extraction
- Built-in tools and MCP servers are configured separately

Setup (GitHub MCP):

1. Choose a supported server in your agent
2. Authenticate and check permissions
3. Enable only the tools you need
4. Test a read-only request

Orchestration: The Big Picture



- Assign distinct tasks and owners. Run independent work in parallel; sequence coupled edits.
- Keep sessions visible and controllable. One machine can use labeled Claude Code and Codex sessions.
- The assistant coordinates projects, machines, inputs, and outputs. A private Command Center can provide this view.
- Match models to tasks and usage limits. Use independent review; link PRs to issues and verify closeout.

Common Frictions in Academic Research

Integration Challenges:

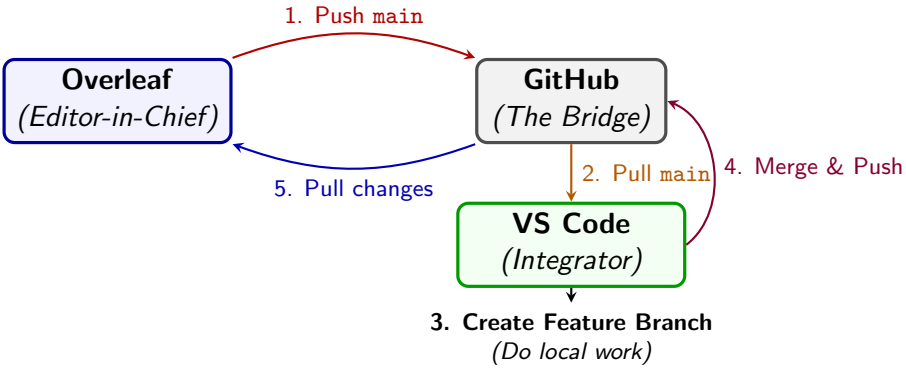
Statistical Software

- **Stata/MATLAB**: Integration depends on extensions and local software
- **Solution**: Run through integrated terminal with commands
- Extensions available for syntax highlighting
- Execute code blocks via terminal shortcuts

Overleaf Integration

- **Via Dropbox**: Sync local folder with Overleaf project (some lag)
- **Via Git**: Clone Overleaf project, push/pull changes (Premium)
- **Via GitHub**: Sync Overleaf project with GitHub repository (requires Premium)
- **Alternative**: Compile locally with LaTeX Workshop; transfer source files when needed

Optimum Workflow: Overleaf + VS Code



Workflow Rules

- **Overleaf** owns `main` (authoritative draft)
- **Local work** happens exclusively on `feature/*` branches
- **Sync** often: Pull `main` before branching

Complementary Research Tools

Tool	Purpose
Refine	AI-powered writing revision and style improvement
(refine.ink)	Interactive editing with suggestions
NotebookLM (Google)	Structured reading and note-taking Create study guides from papers Create academic podcasts!
Elicit	Literature discovery and synthesis Extract data from papers

Integration: Use alongside Git/GitHub workflow for complete research pipeline

Author Instructions: Write for the Paper's Reader

Start with the Argument

- Establish the claims, interpretation, and exhibits before drafting prose
- State the economic argument directly for the intended reader
- Verify findings and citations; never invent results

Reusable Writing Skills

- `academic-draft-editing`
- `econ-write`, `humanizer`

Give Explicit Author Instructions

- Match the existing prose; make minimal edits
- Avoid coined terms, stock trailing clauses, and contrastive padding
- Keep chat history and internal workflow out of the paper
- Introduce notation only when necessary; define every symbol
- Recheck the prose: models can return to their default style

Next Steps & Resources

Next Steps:

1. Fork/clone the workshop repository
2. Install VS Code and extensions
3. Set up GitHub and one AI agent
4. Practice basic Git commands
5. Run `make all` to build the template
6. Apply the workflow to a small project

`https://github.com/tlarroucau/AI_workshop`

Documentation:

- Git: `git-scm.com/doc`
- GitHub: `docs.github.com`
- VS Code: `code.visualstudio.com/docs`

Learning:

- `skills.github.com`
- `software-carpentry.org`

Questions?

Thank you for attending!

Contact: Tomas.Larroucau@asu.edu

Template Repository:
`github.com/tlarroucau/AI_workshop`

The Makefile Approach

What is a Makefile?
A recipe file that automates your workflow. You define *targets*, their *dependencies*, and the *commands* to run.

Syntax

```
target: dependencies
    command to run
```

Then type `make target` in the terminal.

Why it matters for AI: When agents modify your code, `make all` re-runs the full pipeline — so you can inspect regenerated outputs after AI-generated changes.

Example

```
DATA = data/raw/sample_data.csv
OPTS = --robust --output output/

all: analysis paper

analysis: $(DATA)
    python scripts/analysis.py $(OPTS)

paper:
    cd tex/paper && latexmk paper

clean:
    rm -rf output/*
```

Demo: `make all`

Back to Project Structure